

Task C - Best Practices Applied

Default Deny

The implementation uses default-deny behavior for application access and device enrollment. Users and devices only gain access when they match an explicit allow policy.

This matters because Zero Trust should not depend on broad network reachability. Access is granted per identity, per application, and per policy decision.

Least Privilege

The Access model separates normal users from administrators:

- 'Testers' are scoped to the self-hosted application.
- 'SaaS-Users' are scoped to the SaaS application.
- 'Admins' are separate from both user groups.

This avoids using one broad allowlist for every application. It also keeps the owner/admin path available without making every tester an admin.

Reusable Identity Groups

The same group pattern is reused across the Access applications. That makes the access model easier to maintain because the operator can add or remove a user from a group rather than editing every application policy individually.

In this lab, the groups are implemented as Cloudflare-native Access groups and reusable policies with email includes. In a production tenant, the same group names could map to identity-provider group claims from Google Workspace, Okta, or another enterprise IdP.

Private Origin Through Cloudflare Tunnel

The self-hosted application uses Cloudflare Tunnel instead of exposing the origin directly to the internet. This is a strong Zero Trust pattern because the origin does not need inbound firewall openings for public users.

Cloudflare becomes the enforcement point before the request reaches the internal service.

Separate SaaS and Self-Hosted Controls With a Common Policy Model

The assignment includes both self-hosted and SaaS access. I used Cloudflare Access as the common policy layer for both.

This is a useful best practice because it reduces identity sprawl. A user should not have one access model for internal apps and a completely different access model for SaaS if the same Zero Trust policy can govern both.

Ordered Gateway Policies

The HTTP filtering requirement depends on policy order. I applied the specific allow policy before the broad block policy:

1. Allow Netflix and YouTube.
2. Block Entertainment.

That rule order is documented at <https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/gateway-policies/README.md>, because changing the order would break the exception.

Policy-as-Code Documentation

The repo stores the important policy definitions as JSON:

- <https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/access-policies/groups.json>
- <https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/access-policies/self-hosted-app.json>
- <https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/access-policies/saas-app.json>
- <https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/gateway-policies/dns-policies.json>
- <https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/gateway-policies/http-policies.json>

This makes the implementation reviewable outside the dashboard and gives a starting point for automation through Terraform, API calls, or a deployment script.

Evidence-Based Deliverables

The written report is backed by screenshots and policy files instead of only prose. The screenshots show the dashboard state, while the JSON files show the intended reusable configuration.

That combination is useful for assignments and real operations because it separates "what was configured" from "why it was configured."

Documented Limitations

I documented the personal Gmail limitation instead of hiding it. Since personal Gmail does not provide Workspace group claims, the assignment uses Cloudflare-native groups and reusable policies with email includes.

That is not the ideal enterprise design, but it is the correct transparent choice for this environment. The production upgrade path is clear: replace email includes with IdP group claims once a Workspace or enterprise IdP is available.